# Assignment 2: High Performance Computer Architecture

EO-243 HPCA

Yashi Agrawal
yashiagrawal@iisc.ac.in
SRNO: 26612

October 21, 2025

## Contents

**1 Objective** **2**

**2 System Specifications** **2**

**3 Collection of Data and Code execution** **2**

3.1 Setting up the system and Configuring the kernel settings and hugepages: . . . . . 2
3.2 Step 1: Building the main executable and getting the report of TLB misses . . . . . 2
3.3 Step 2: Parsing the small pages `smallpages.txt` . . . . . 3
3.4 Step 3: Running `analyze.py` to get `hugepages.txt` . . . . . 3
3.5 Step 4: Run modified `main.c` that reads `hugepages.txt` . . . . . 3
3.6 Step 5: Analyze `modified_main.c` with perf mem . . . . . 3
3.7 Step 6 : Run `parse_large_pages.py` on `largepages.txt` . . . . . 3

**4 Results and Analysis** **4**

4.1 Data Analysis: TLB miss distribution before vs after . . . . . 4
4.2 Number of Huge pages deployed . . . . . 6
4.3 Performance Improvement . . . . . 6

**5 Conclusion** **7**

# 1 Objective

To implement 2MB huge pages, in place of 4KB small pages at virtual address regions with high TLB misses, and analyze the improvement in performance obtained.

# 2 System Specifications

1. **CPU:** core i7
2. **Virtual address size:**48 bits virtual address
3. **CPU clock speed**3.20 GHz
4. **L1d cache:** 192KiB
5. **L1i cache:** 192KiB
6. **L2 cache:** 1.5MiB
7. **L3 cache:** 12MiB

# 3 Collection of Data and Code execution

**Profiling tool:**perf mem

We first run the perf mem profiling tool to get the virtual address points of the various TLB misses that occur for the sampled data. We then compute the 2MiB regions for the addresses obtained and calculate the frequency of misses that occur in these events. Once this is done,, we allocate N huge pages to these 2MiB regions, and then analyze the improvement in performance obtained due to the same.

## 3.1 Setting up the system and Configuring the kernel settings and hugepages:

If the system does not have perf installed, install using:

```
sudo apt-get install linux-tools-common linux-tools-generic linux-tools-$(uname -r)
```

Configure kernel settings and huge pages with the commands:

```
sudo sysctl vm.nrhugepages=512
sudo sysctl kernel.perfevent_paranoid-1
```

## 3.2 Step 1: Building the main executable and getting the report of TLB misses

We run the following command with our SR number, 26612 in my case, and run the following command, followed with the perf mem command to create the report of the TLB hits and misses alongside their addresses

To make the file executable:
```
sudo make all SRNO=26612
```

Now we can monitor the TLB misses as well as the virtual addresses of the TLB misses via this instruction, this will also help in recording the time of execution of the code:
```
sudo perf mem record ./main 26612
```

Finally, we can create the report and save it in a text file from perf data that was obtained using the previous command via using this command:
```
sudo perf mem report> smallpages.txt
```

### 3.3 Step 2: Parsing the small pages `smallpages.txt`

We now run the `parse_small_pages.py` file to parse the `smallpages.txt` file. This will parse the addresses of TLB misses , and convert the 4KiB regions to 2MiB regions, while counting the frequency of the samples so as to calculate and return the frequency of TLB misses occurring in each 2MiB region in descending order. This helps us in deciding which regions do we want to allocate huge pages to. We get the file `smallpages_parsed.txt`

### 3.4 Step 3: Running `analyze.py` to get `hugepages.txt`

We run `analyze.py` on `smallpages_parsed.txt` to get the N addresses where we will be allocating huge pages and then return the addresses of those huge pages in decimal format.

### 3.5 Step 4: Run modified `main.c` that reads `hugepages.txt`

We now run the modified `main.c` file, by modifying the `main.c` file such that it reads through the `hugepages.txt` file by means of a while loop to allocate huge pages to the addresses in the `hugepages.txt` file.

We use the following code to run `modified_main.c`:
```
sudo make all SRNO=26621
```

### 3.6 Step 5: Analyze `modified_main.c` with perf mem

We will now analyze the performance improvement by running the `modified_main.c` using perf mem for profiling. We use the following code in terminal:

```
sudo perf mem record ./modified_main 26621
sudo perf mem report> largepages.txt
```

### 3.7 Step 6 : Run `parse_large_pages.py` on `largepages.txt`

We now run the `parse_large_pages.py` on `largepages.txt` to get the frequency of TLB misses in the 2MiB regions ,that are being calculated by parsing `largepages.txt`.

# 4 Results and Analysis

Upon the execution of the above mentioned program flow upto step 3, we get the hugepages.txt file, that contains the frequency of TLB misses for the different 2MiB regions, we have taken all the 2MiB regions with frequency grater than equal to 12, i.e, we have allotted a total of 471 huge pages.

## 4.1 Data Analysis: TLB miss distribution before vs after

We can see the TLB misses for the different 2MiB regions before and after the allocation of huge pages in Figure 1 and Figure 2 respectively.

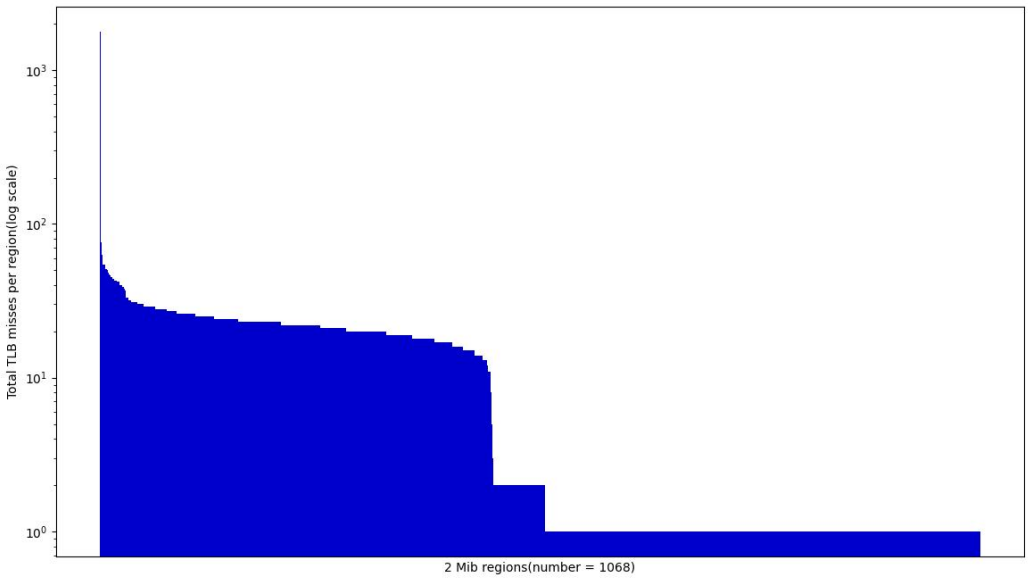


Figure 1: TLB Misses per 2MB Region Before Huge Page Allocation. The x-axis represents the base address of a 2MB region, and the y-axis is the number of sampled TLB misses

It can be noticed, we notice a huge amount of TLB misses before the allocation of huge pages, in contrast the number of TLB misses post the allocation of huge pages is significantly low.

This is a significant observation that visually stresses upon the fact that the allocation of huge pages had such a significant impact on TLB misses, thus performance increases greatly.

We can see the comparison graph of TLB misses , before and after allocation of huge pages in a side by side manner in Figure 3.

Figure 4 shows the bar graph comparison for the top 20 common hotspot regions.

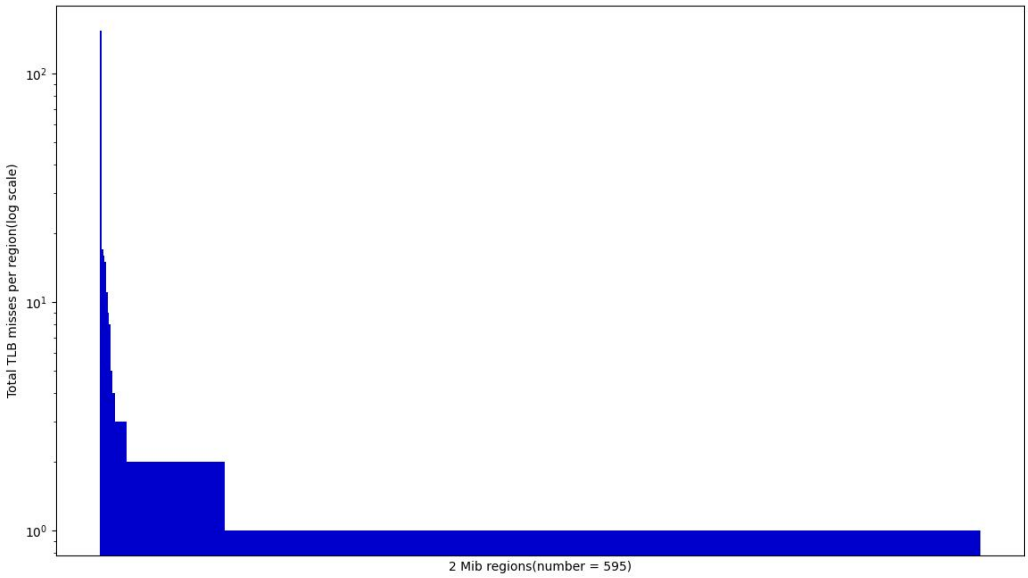


Figure 2: TLB Misses per 2MB Region After Huge Page Allocation. The x-axis represents the base address of a 2MB region, and the y-axis is the number of sampled TLB misses :

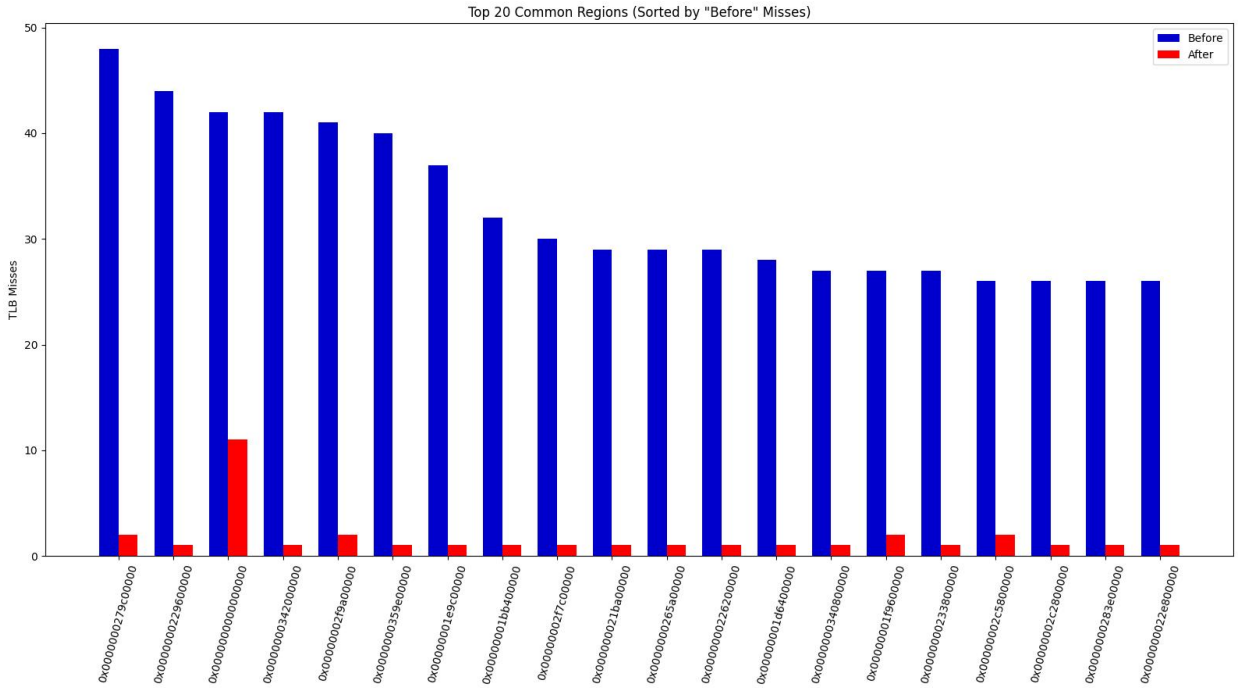


Figure 3: TLB Misses of top 20 common 2MB Region After Huge Page Allocation. The x-axis represents the base address of a 2MB region, and the y-axis is the number of sampled TLB misses, the blue bar graph containes before value of TLB misses and red bar contains the after value :

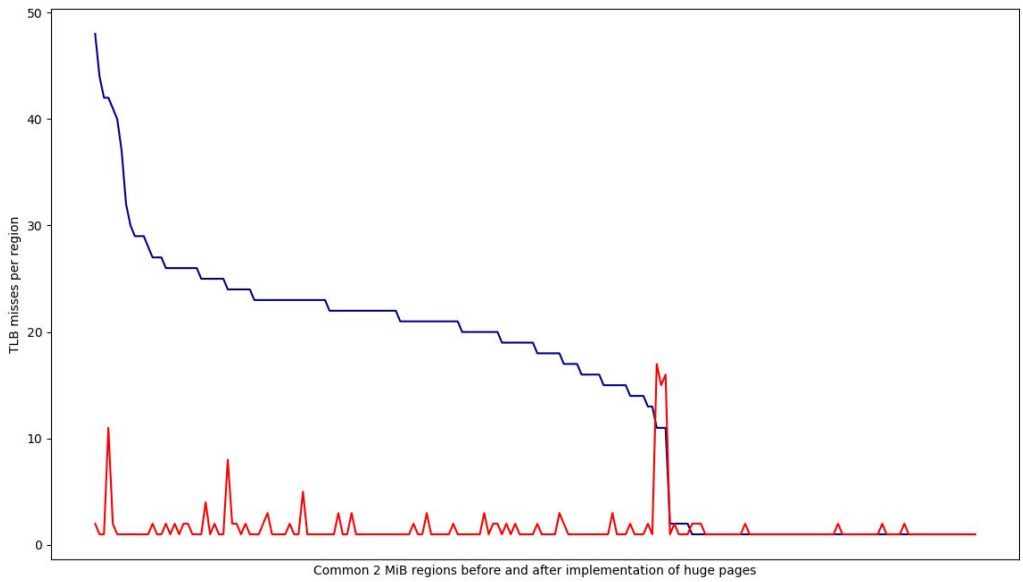


Figure 4: TLB Misses per 2MB Region Before and After Huge Page Allocation. The x-axis represents the common base addresses of 2MB regions, and the y-axis is the number of sampled TLB misses:

### 4.2 Number of Huge pages deployed

We have a total of 471 huge pages deployed. We have taken all the hugepages that have page faults greater than equal to 12 and hence the number of huge pages has been taken as 471. The assignment had a budget of huge pages, implemented by means of cgroups, and thus a number close enough to the boundary was taken for the deployment of huge pages.

### 4.3 Performance Improvement

The final performance improvement is measured by comparing the execution time and total TLB misses of the original program against the optimized (N hugepages allocated) program

| **Metric** | **Original** | **Optimized (N Pages)** |
|---|---:|---:|
| Execution Time (sec) | 12.10 | 9.89 |
| Total TLB Misses | 13503 | 910 |

Table 1: Performance Comparison Before and After Huge Page Allocation.

1. **Speedup:** The speedup is calculated as the ratio of original execution time to optimized execution time.

$$\text{Speedup} = \frac{\text{Time}_{\text{original}}}{\text{Time}_{\text{optimized}}} = \frac{12.10}{9.89} = \mathbf{1.22}$$

2. **Reduction of TLB Misses:** The percentage reduction in TLB misses is calculated as:

$$\text{Reduction } \% = \frac{\text{Misses}_{\text{original}} - \text{Misses}_{\text{optimized}}}{\text{Misses}_{\text{original}}} \times 100$$

$$\text{Reduction } \% = \frac{13503 - 910}{13503} \times 100 = \mathbf{93.3}\%$$

Thus, we notice a speedup of **1.22** as well as a **93.3%** reduction in TLB misses, this shows a clear increase in performance. we can see that the TLB misses have reduced by a very huge percentage and we are also able to see a significant speedup.

This proves that the assignment of huge pages clearly impacts the performance of a program in a positive way.

# 5 Conclusion

After implementing huge pages , we have obtained a speedup of 1.22x and a 93.3% reduction in the TLB misses,this indicates we have a significant performance boost with the implementation of huge pages.